Heathkit H8 für Dummies

Oct 20, 2026 · @Roland Kirschning

Ein Einsteigerbuch auf Deutsch: vom ersten Blick auf die Frontplatte bis zum eigenen Programm in BASIC und Assembler, mit Beispielen zum Nachmachen am Original oder im Emulator.



Der H8 in der Emulation: Frontplatte mit Lampen, Adress- und Datenanzeige und dem Tastenfeld, das du in Kapitel 3 kennenlernst (Seite 1 von The Final H8/H9).

Vorwort

Dieses Buch bringt dir den Heathkit H8 bei, ohne dass du vorher je einen 8080 programmiert haben musst. Du lernst den Rechner kennen, tippst dein erstes Programm über die Frontplatte ein und schreibst danach eigene Programme in BASIC und Assembler.

Für wen ist das Buch?

Für Einsteiger, Bastler und Retro-Fans, die einen H8 besitzen, ihn restaurieren wollen oder ihn im Emulator erkunden möchten. Vorkenntnisse in Elektronik sind nicht nötig. Du solltest wissen, was ein Bit und ein Byte sind. Alles Weitere erklärt das Buch.



*Das H9 Video Terminal in der Emulation „The Final H8/H9“: Extended Benton Harbor BASIC hat ein kleines Programm ausgeführt und wartet am *-Prompt.*

Was brauchst du?

Es gibt zwei Wege, die Beispiele nachzuvollziehen:

- **Original-Hardware:** ein H8 mit mindestens einer Speicherkarte, einem Terminal (zum Beispiel H9 oder H19) und, für die späteren Kapitel, einer Kassettenschnittstelle oder einem H17-Diskettenlaufwerk.
- **Emulator:** ein Programm, das den H8 am Computer nachbildet. Kapitel 11 nennt Möglichkeiten, darunter den Browser-Emulator „The Final H8/H9“, der ohne Installation läuft. Fast alle Beispiele laufen dort identisch, weil Emulatoren dasselbe ROM und dieselben Tastenabläufe verwenden.

So ist das Buch aufgebaut

Die Kapitel bauen aufeinander auf. Kapitel 1 bis 3 erklären, was der H8 ist und wie seine Frontplatte funktioniert. In Kapitel 4 tippst du das erste Programm ein. Kapitel 5 und 6 verbinden den Rechner mit Terminal und Kassette. Kapitel 7 und 8 sind der

Programmierteil mit BASIC und Assembler. Kapitel 9 behandelt Disketten und das Betriebssystem HDOS. Kapitel 10 hilft bei Fehlern, Kapitel 11 zeigt den H8 heute. Der Anhang enthält Tabellen zum Nachschlagen.

Schreibweisen

Zahlen in **Oktal** (Basis 8) tragen den Zusatz `Q`, zum Beispiel `370Q`. Das ist die Schreibweise, die Heath in seinen Handbüchern nutzt. Adressen in der Frontplatte werden in zwei Dreiergruppen gelesen, etwa `040 100`. Das nennt man **Split-Octal**. Tastenfolgen stehen in eckigen Klammern, zum Beispiel [GO].

Kapitel 1 – Was ist der Heathkit H8?

Der H8 ist ein Computer mit Intel-8080A-Prozessor, den die Firma Heath ab 1977 als Bausatz verkaufte. Man lötete ihn selbst zusammen und bekam dafür ausführliche Handbücher, die jeden Schritt erklärten.

Kurze Geschichte

Heath, in Benton Harbor (Michigan) zu Hause, baute seit Jahrzehnten Elektronik-Bausätze. Nach dem Erfolg des Altair 8800 entschied sich die Firma 1977 für einen eigenen Computer. Angekündigt wurde der H8 im Juli 1977, in den Handel kam er im Herbst desselben Jahres. Der Bausatz kostete 379 US-Dollar.

Die Zeitgenossen unterschieden den H8 vom Altair durch drei Dinge: ein sauber entworfener Bus, eine Frontplatte mit Ziffernanzeige statt Kippschaltern und ein eingebautes Monitorprogramm im ROM. Zwei Jahre später erschien mit dem H89 ein Komplettgerät aus Terminal, Rechner und Diskettenlaufwerk, das später unter dem Namen Zenith Z-89 verkauft wurde. 1979 übernahm Zenith die Firma.

Was war im Karton – und was nicht?

Das ist die wichtigste Lektion für jeden Einsteiger: **Der H8-Grundbausatz allein tut nichts Nützliches.** Er enthielt Gehäuse, Netzteil, Frontplatte und CPU-Karte, aber keinen Arbeitsspeicher. Mindestens eine Speicherkarte musste dazukommen. Ohne Terminal blieb nur die Frontplatte zur Bedienung.

Baugruppe	Aufgabe	Preis damals
H8 (Grundgerät)	Gehäuse, Netzteil, Frontplatte, CPU mit 8080A und Monitor-ROM	379 US-Dollar
H8-1	Speicherkarte mit 4 KByte statischem RAM	140 US-Dollar

Baugruppe	Aufgabe	Preis damals
H8-3	Chipsatz, erweitert eine H8-1 auf 8 KByte	95 US-Dollar
H8-5	Serielle Schnittstelle plus Kassetteninterface	110 US-Dollar
H8-2	Parallele Schnittstelle mit drei Ports	150 US-Dollar
H9	Video-Terminal mit Tastatur	530 US-Dollar

Ein lauffähiges System aus Grundgerät, Speicher, Serienkarte und Terminal kostete also schnell das Dreifache des Grundpreises.

Die Idee hinter dem H8

Heath wollte einen Rechner, den Hobbyisten verstehen. Die Frontplatte zeigt Adressen und Daten in **Oktalzahlen**, einer Schreibweise, die sich leichter in 8-Bit-Bytes übersetzen lässt als Binärzahlen. Das Monitorprogramm **PAM-8** im ROM erlaubt es, Programme von Hand einzugeben, zu starten und Schritt für Schritt zu verfolgen. Ein Rechner, der sich selbst erklärt, so war der Gedanke.

Was du aus diesem Buch mitnimmst

Du verstehst am Ende, wie ein Rechner dieser Generation funktioniert. Das hilft auch, wenn du heute einen 8080 oder Z80 in einem Emulator oder auf einer Retro-Platine programmierst.

Kapitel 2 – Die Hardware im Überblick

Der H8 besteht aus einem Gehäuse mit Netzteil und einer Rückwand-Platine, in die du Steckkarten einschiebst. Frontplatte, CPU, Speicher und Schnittstellen sind jeweils eigene Karten.

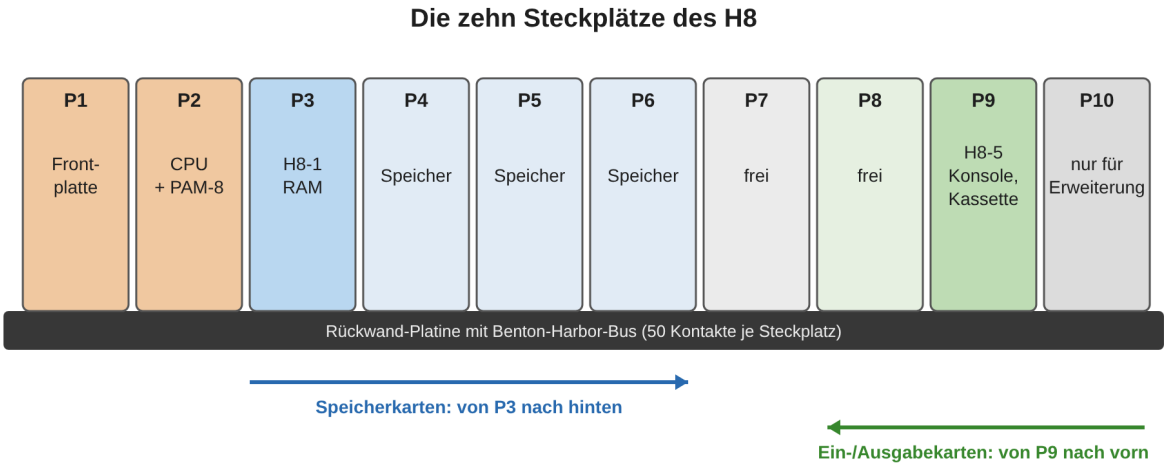
Das Gehäuse und der Bus

Heath verwendete keinen S-100-Bus wie der Altair, sondern einen eigenen mit 50 Kontakten. Er heißt nach der Heimatstadt der Firma **Benton-Harbor-Bus**. Er führt 16 Adressleitungen, 8 Datenleitungen, fünf Interruptleitungen und die Steuersignale. Zehn Steckplätze stehen zur Verfügung.

Steckplatz	Belegung
P1	Frontplatte

Steckplatz	Belegung
P2	CPU-Karte (immer hier einstecken)
P3 und folgende	Speicherkarten, von P3 in Richtung Rückseite
P9 und davor	Ein-/Ausgabekarten, von P9 in Richtung Vorderseite
P10	nur für Erweiterungen vorgesehen, nicht bestücken

Das Handbuch empfiehlt, Karten in jedem zweiten Steckplatz einzusetzen, weil der H8 nur durch Konvektion gekühlt wird. Die Lüftungsschlitze oben und unten dürfen nicht verdeckt werden.



Die Steckplätze P1 bis P10 im Überblick. Die Beispielkarten zeigen eine mögliche Bestückung.

Netzteil

Das Netzteil liefert unstabilisierte Spannungen: +8 Volt mit bis zu 10 Ampere sowie ±18 Volt mit 250 Milliampere. Jede Karte erzeugt sich ihre +5 Volt mit eigenen Spannungsreglern. Am Gehäuse sitzt ein Schalter LOW/NORM für die Netzspannung. Er bleibt auf NORM, solange du nicht sicher bist, dass deine Netzspannung niedrig ist.

> Wichtig: Karten nie bei eingeschaltetem Gerät einstecken oder entfernen.

Die CPU-Karte

Auf der CPU-Karte arbeitet ein **Intel 8080A**. Ein Taktgeber liefert zwei nicht überlappende Takte mit 2,048 MHz. Der Prozessor kann 64 KByte adressieren und hat sieben Interruptebenen. Auf der Karte sitzt außerdem das ROM mit dem Monitor **PAM-8**. In der ersten Ausführung ist es 1 KByte groß. Später verkaufte Heath größere ROMs mit mehr Funktionen.

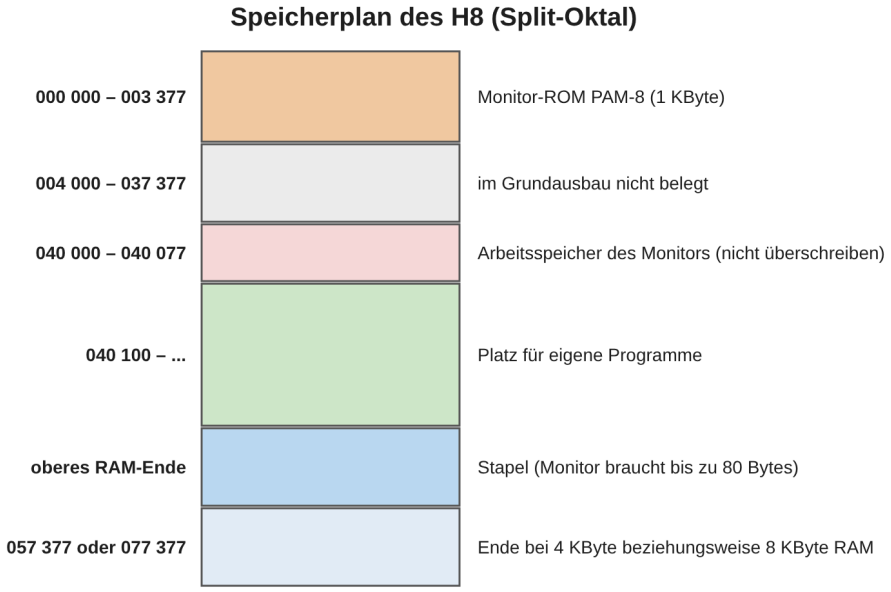
Beim Einschalten prüft der Monitor selbständig, wie viel zusammenhängender Speicher vorhanden ist, und legt den Stapelzeiger an das obere Ende. Das siehst du später im Register SP.

Speicher

Die H8-1 trägt 4 KByte statisches RAM, mit dem Chipsatz H8-3 wächst sie auf 8 KByte. Bis zu vier Karten sind möglich, zusammen 32 KByte. Das RAM beginnt bei Adresse 8K, in Split-Oktal **040 000**.

Adresse (Split-Oktal)	Inhalt
000 000 – 003 377	Monitor-ROM PAM-8 (1 KByte)
040 000 – 040 077	Arbeitsspeicher des Monitors, nicht überschreiben
040 100	Hier beginnen eigene Programme
oberes RAM-Ende	Stapel, der Monitor braucht dort bis zu 80 Bytes

Bei einer 4-KByte-Karte endet der Speicher bei 057 377, bei einer 8-KByte-Karte bei 077 377.



Die Höhen der Blöcke sind nicht maßstäblich.

Der Speicherplan: ROM unten, ab 040 000 das RAM mit dem Arbeitsbereich des Monitors, darüber deine Programme.

Ports für Ein- und Ausgabe

Neben dem Speicher kennt der 8080 bis zu 256 Ein- und Ausgabe-Ports. Die wichtigsten Adressen des H8, alle in Oktal:

Port	Verwendung
360Q	Frontplatte: Befehle, Ziffernauswahl, Tastenfeld
361Q	Frontplatte: Segmentauswahl
370Q / 371Q	Laden und Sichern (Kassette): Daten / Steuerung
372Q / 373Q	Konsole (Terminal): Daten / Steuerung
350Q – 357Q	serielle Karte H8-4 (ab 350Q sind Ports für Systemsoftware reserviert)
174Q – 177Q	Diskettensteuerung H17

Die H8-5 enthält zwei serielle Bausteine vom Typ 8251: einen für die Konsole, einen für die Kassette. Ältere Ausführungen der Karte trennen die Leitungen optisch, was sie auf niedrige Geschwindigkeiten begrenzt. Die später erschienene Karte H8-4 mit mehreren Ports und das Terminal H19 führten 9600 Baud als Standard ein.

Kapitel 3 – Die Frontplatte und der Monitor PAM-8

Die Frontplatte ist dein Schreibtisch am H8: ein Tastenfeld mit 16 Tasten, neun Ziffernanzeigen und vier Kontrolllampen. Der Monitor PAM-8 im ROM wertet die Tasten aus und steuert die Anzeige.

Lampen, Anzeige und Piepton

Lampe	Bedeutung
ION	der Prozessor nimmt Interrupts an
MON	du hast Kontrolle über die Frontplatte (Monitor-Modus)
RUN	der Prozessor arbeitet ein Programm ab
PWR	die +5 Volt der Frontplatte sind da

Die neun Ziffern sind in drei Dreiergruppen geteilt. Die linken sechs zeigen die Adresse (zuerst das höhere, dann das niedrige Byte), die rechten drei zeigen Daten oder den Namen eines Registers. Alle Zahlen erscheinen in Oktal.

Der eingebaute Lautsprecher bestätigt jede Eingabe: ein **kurzer** Ton für jeden Tastendruck, ein **mittlerer** Ton, wenn eine Dreiergruppe vollständig ist, und ein **langer** Ton bei Reset oder ungültigem Befehl. Gerade bei langen Programmen ist das Gold wert, weil du am Ton hörst, ob die Eingabe angekommen ist.

Split-Oktal richtig lesen

Ein Byte hat acht Bit. In Oktal zeigt der H8 es als drei Ziffern. Die linke Ziffer deckt nur zwei Bit ab, deshalb ist sie höchstens 3. Das größte Byte ist also **377**.

Eine 16-Bit-Adresse wird als zwei Bytes angezeigt, etwa `040 100`. Das ist kein echtes 6-stelliges Oktal. Im Split-Oktal folgt auf `000 377` die Adresse `001 000`. Behalte das im Kopf, sonst rechnest du dich bei Sprungzielen schnell aus der Bahn.

Die Tasten

Die Funktion jeder Taste steht klein unten rechts auf der Taste. Die Ziffern 0 bis 7 geben Oktalzahlen ein.

Funktion	Wirkung
MEM	Speichermodus: Adresse eingeben, Inhalt ansehen

Funktion	Wirkung
REG	Registermodus: Register auswählen und ansehen
ALTER	Inhalt ändern, Taste erneut zum Verlassen
+ und −	nächste oder vorige Adresse beziehungsweise nächstes Register
CANCEL	aktuelle Eingabe abbrechen (langer Ton)
GO	Programm ab dem Programmzähler starten
SI	genau einen Befehl ausführen
LOAD und DUMP	von Kassette laden oder darauf sichern
IN und OUT	einen Port lesen oder beschreiben

Zwei Funktionen erfordern **zwei Tasten gleichzeitig**, damit du sie nicht aus Versehen auslöst. Die Taste ø zusammen mit RST löst einen Reset aus (alle Register gehen verloren). Die Taste ø zusammen mit RTM bringt dich aus einem laufenden Programm zurück in den Monitor, ohne Registerinhalte zu zerstören.

In der Emulation gibt es dafür Tastenkürzel: Reset (RST/0) mit Cmd + 0 und RTM mit Umschalt + 0, jeweils kurz gedrückt gehalten. Der Reset löscht dort auch das RAM.

Hältst du eine Taste gedrückt, wiederholt sie sich ungefähr alle 400 Millisekunden. Das ist praktisch zum schnellen Durchblättern des Speichers.

Die Dezimalpunkte als Statusanzeige

Die Punkte hinter den Ziffern sagen dir, was der Monitor als Nächstes von dir erwartet:

Dezimalpunkte	Bedeutung
alle leuchten ständig	bereit, eine Speicheradresse einzugeben
linke sechs leuchten ständig	bereit, ein Register zu wählen
alle laufen von rechts nach links	bereit, Speicherinhalt zu ändern
linke sechs laufen	bereit, Registerinhalt zu ändern

Der Speicher ansehen und ändern

1. Drücke MEM. Alle Punkte leuchten.

2. Gib sechs Ziffern ein, zum Beispiel `040123`. Nach der sechsten erlöschen die Punkte. Links siehst du die Adresse, rechts den Inhalt.
3. Drücke ALTER. Die Punkte laufen.
4. Tippe drei Ziffern, zum Beispiel `345`. Der Wert ist geschrieben, die Adresse springt eine Stelle weiter.
5. Drücke − und du siehst deinen Wert an der vorigen Adresse. Mit ALTER verlässt du den Änderungsmodus.

Tippst du dich vertan, drücke − und gib die drei Ziffern neu ein. Das funktioniert nur im RAM. Im ROM bleibt der Inhalt unverändert.

Register ansehen

Drücke REG und danach eine Ziffer: **1** = SP (Stapelzeiger), **2** = AF, **3** = BC, **4** = DE, **5** = HL, **6** = PC (Programmzähler). Der Registerpaarname erscheint rechts, die Inhalte stehen links, je drei Ziffern pro Register. Mit + und − wanderst du durch die Paare. Mit ALTER änderst du einen Wert. Den Stapelzeiger kannst du nicht direkt ändern, dafür brauchst du einen kleinen Umweg über den Befehl SPHL.

Programme starten und anhalten

Du setzt den Programmzähler (PC) auf die erste Adresse und drückst GO. Mit SI führst du nur einen einzelnen Befehl aus, ideal zur Fehlersuche. Das klappt am echten H8 und in der Emulation. Ein HLT-Befehl (Oktalcode 166) im Programm ruft den Monitor zurück und lässt den Lautsprecher ertönen: So setzt du einen **Haltepunkt**. Der Programmzähler zeigt dann auf die Adresse nach dem HLT. Willst du später weiterlaufen, ersetze HLT durch NOP (Code 000).

Zwei Regeln, die dich vor Ärger bewahren

- Der Monitor braucht ungefähr 10 Prozent der Rechenzeit und braucht am oberen Speicherende mindestens 80 freie Bytes für den Stapel. Zerstörst du den Stapel, stürzt dein Programm ab.
- Führt dein Programm den Befehl DI aus (Interrupts aus), reagiert die Frontplatte nicht mehr, bis ein EI folgt oder du einen Reset auslöst.

Kapitel 4 – Praxis: das erste Programm per Frontplatte

In diesem Kapitel tippst du vier kleine Maschinenprogramme ein. Alle liegen ab Adresse 040 100, dem ersten freien RAM-Platz hinter dem Arbeitsbereich des Monitors. Wenn du keinen echten H8 hast, probiere sie im Browser-Emulator „The Final H8/H9“ aus. Er enthält den echten Monitor PAM-8. Auf Seite 1 findest du die Nachbildung der Frontplatte.

Das Eingabeverfahren

Jedes Programm folgt demselben Ablauf:

1. **Adresse wählen:** MEM, dann `040100`.
2. **Bytes eintragen:** ALTER drücken, dann jedes Byte als drei Ziffern tippen. Die Adresse zählt von selbst weiter. Zum Schluss ALTER noch einmal drücken.
3. **Prüfen:** MEM, `040100`, dann mit + Byte für Byte durchblättern.
4. **Programmzähler setzen:** REG, **6** (PC), ALTER, `040100`, ALTER.
5. **Starten:** GO.

Reset und RTM sind nicht dasselbe. In der Emulation löscht **RST/0** (am Mac Cmd + 0) das gesamte RAM, dein Programm ist danach weg. Ein echter H8 würde das RAM beim Reset nicht leeren, die Emulation tut es, damit der Monitor sauber startet. **RTM** (am Mac Umschalt + 0) dagegen hält ein laufendes Programm nur an und holt den Monitor zurück. Das Programm bleibt im Speicher. Nimm RST/0 also vor einem neuen Programm und RTM, wenn du ein laufendes Programm stoppen und behalten willst.

Programm 1: Zwei Zahlen addieren (7 + 5)

Adresse	Bytes	Befehl	Wirkung
040 100	076 007	MVI A,7	Register A = 7
040 102	006 005	MVI B,5	Register B = 5
040 104	200	ADD B	A = A + B
040 105	166	HLT	Halt, Rückkehr in den Monitor

Nach GO ertönt der Lautsprecher, weil der Monitor zurückkehrt. Der Programmzähler steht auf 040 106. Drücke REG und **2:** In den linken drei Ziffern steht jetzt **014**. Das ist 12 dezimal (1·8 + 4). Daneben siehst du das Flag-Register.



Programm 1 in der Emulation, von links oben nach rechts unten: erstes Byte 076 an Adresse 040 100, Programmzähler gesetzt, nach GO steht der Programmzähler auf 040 106, REG 2 zeigt im Akkumulator 014.

Zeigt dein Register etwas anderes? Dann drücke vor dem Eintippen immer **RST/0**. Der Reset löscht die geladenen Programme im RAM und setzt die Register zurück. Sonst können Reste eines früheren Programms (zum Beispiel BASIC) im Akkumulator stehen, und du siehst statt `014` einen anderen Wert wie `116`. Das Register F zeigt bei diesem Beispiel `006`, die Anzeige lautet also `014 006`. Prüfe außerdem, ob der Programmzähler vor GO auf `040 100` steht und nach GO auf `040 106`.

Programm 2: Ein Zähler bis 100

Adresse	Bytes	Befehl	Wirkung
040 100	076 000	MVI A,0	A = 0
040 102	074	INR A	A = A + 1
040 103	376 144	CPI 144	A mit 100 dezimal (144 oktal) vergleichen
040 105	302 102 040	JNZ 040 102	zurück zu INR A, solange A nicht 100 ist
040 110	166	HLT	fertig, Monitor kommt zurück

Das Programm zählt A von 0 an hoch und vergleicht nach jedem Schritt mit `CPI` den Wert mit 144 oktal, das sind 100 dezimal. Solange A nicht 100 ist, springt `JNZ` zurück. Dann trifft der Rechner auf `HLT` und hält an. Starte mit GO. Nach einem Moment leuchtet MON, und der Programmzähler steht auf `040 111`. Wähle mit REG und **2** das Registerpaar AF: Im Akkumulator steht `144`, also 100 dezimal. Rechts daneben zeigt das Register F meist `126`. Das sind die Flaggen, die der letzte Vergleich gesetzt hat.

Die Schleife läuft so schnell, dass du den Zähler nicht mitzählen kannst. Das Ergebnis siehst du erst am Ende. Mit SI kannst du ihr aber Befehl für Befehl zusehen, wie die folgende Übung zeigt.

Programm 2 als Endlosschleife: Schritt für Schritt mit SI

Ersetze den Vergleich durch einen unbedingten Sprung (`JMP`). Dann zählt das Programm endlos weiter und hält nie von selbst an. Drücke vorher RST/0 und tippe ab 040 100:

Adresse	Bytes	Befehl	Wirkung
040 100	076 000	MVI A,0	A = 0

Adresse	Bytes	Befehl	Wirkung
040 102	074	INR A	A = A + 1
040 103	303 102 040	JMP 040 102	immer zurück zu INR A

1. Starte mit GO. Die Anzeige RUN leuchtet, das Programm zählt endlos.
2. Stoppe es mit RTM (Mac: Umschalt + 0 kurz gedrückt halten). MON leuchtet, das Programm bleibt im Speicher.
3. Wähle mit REG und **2** das Registerpaar AF. In A steht irgendein Wert, je nachdem, wann du gestoppt hast.
4. Drücke jetzt mehrmals SI (in der Emulation Taste 7). Jeder Druck führt genau einen Befehl aus. A wächst bei jedem zweiten Druck um 1: Einmal läuft `INR A`, einmal `JMP`.
5. Wähle mit REG und **6** den Programmzähler. Bei jedem SI wechselt er zwischen `040 102` und `040 103`.

Mit GO läuft das Programm danach wieder in voller Geschwindigkeit weiter. Diese Übung habe ich im Emulator geprüft.

Programm 3: Die Summe 1 bis 10

Adresse	Bytes	Befehl	Wirkung
040 100	006 012	MVI B,10	Zähler B = 10
040 102	076 000	MVI A,0	Summe A = 0
040 104	200	ADD B	A = A + B
040 105	005	DCR B	B = B − 1
040 106	302 104 040	JNZ 040 104	wiederholen, solange B nicht 0
040 111	166	HLT	fertig

Die Zeile `006 012` meint die Zahl 10 dezimal, die in Oktal 012 lautet. Das Ergebnis in A ist **067**, also 55 dezimal. Das ist die erste echte Schleife: Das Register B zählt herunter, der Sprung JNZ springt zurück, bis B null erreicht.

Programm 4: Einen Speicherbereich füllen

Adresse	Bytes	Befehl	Wirkung
040 100	041 200 040	LXI H,040 200	HL zeigt auf Ziel 040 200

Adresse	Bytes	Befehl	Wirkung
040 103	006 020	MVI B,16	16 Bytes
040 105	076 377	MVI A,377	Füllwert 377
040 107	167	MOV M,A	Wert an Adresse in HL schreiben
040 110	043	INX H	HL = HL + 1
040 111	005	DCR B	Zähler herunter
040 112	302 107 040	JNZ 040 107	weiter, bis 16 geschrieben
040 115	166	HLT	fertig

Nach dem Lauf schaust du mit MEM, `040200` und + in den Speicher: 16 Bytes zeigen 377, das 17. bleibt unverändert. Achte darauf, dass im Programm die Adresse im Befehl LXI zuerst das niedrige Byte (200) und dann das hohe Byte (040) enthält. So speichert der 8080 jede 16-Bit-Zahl.



Links: Nach Programm 3 zeigt REG 2 im Akkumulator 067. Rechts: Nach Programm 4 steht an Adresse 040 200 der Wert 377.

Wenn etwas nicht klappt

- **Langer Ton bei einer Taste:** Das Gerät erwartet etwas anderes. Drücke CANCEL und beginne den Schritt neu.
- **Programm läuft nicht an:** Prüfe mit REG und **6**, ob der Programmzähler wirklich auf 040 100 steht.
- **Falsches Ergebnis:** Blättere die Bytes noch einmal durch. Ein verdrehter Ziffernsturz fällt dort sofort auf.
- **Totalabsturz:** ø + RST löst einen Reset aus, danach beginnst du neu.

Kapitel 5 – Ein Terminal anschließen

Mit der Frontplatte allein bleibt der H8 eine Übungsmaschine. Ein Terminal macht daraus einen richtigen Rechner: Du tippst Text ein und siehst Antworten auf dem Bildschirm.

Welche Terminals passen?

- **H9:** das erste Heath-Terminal, nur Großbuchstaben und 12 Zeilen zu je 80 Zeichen. Es verarbeitet vom Rechner nur wenige Steuerzeichen und bietet keine Cursor-Steuerung über Escape-Folgen.
- **H19:** das spätere, deutlich komfortablere Terminal mit Kleinbuchstaben und Steuerfolgen.
- **Jedes andere serielle Terminal**, etwa ein PC mit Terminalprogramm und USB-Seriell-Adapter.

Angeschlossen wird es an die Karte **H8-5** (oder an die H8-4). Die Geschwindigkeit stellst du an der Karte ein und am Terminal auf denselben Wert.

Die Konsole in zwei Ports

Das Terminal hängt an den Ports 372Q und 373Q. Beide gehören zu einem Baustein vom Typ 8251:

Port	Lesen	Schreiben
373Q	Status	Steuerbefehle (Modus, Kommando)
372Q	empfangenes Zeichen	zu sendendes Zeichen

Im Statuswort sind zwei Bits wichtig: **Bit 0** (Wert 001) meldet, dass der Sender ein neues Zeichen annehmen kann. **Bit 1** (Wert 002) meldet, dass ein Zeichen empfangen wurde und bereit liegt.

Programm 5: Konsole einrichten und Text ausgeben

Der 8251 meldet erst dann „Sender bereit“, wenn er eingerichtet ist. Das Programm besteht deshalb aus zwei Teilen: Zuerst die Einrichtung (Schreibzugriffe auf den Steuerport 373Q), danach die Ausgabe, die Zeichen für Zeichen aus dem Speicher holt und sendet, jeweils erst nach Prüfung des Statusbits. Ohne den ersten Teil würde das Programm endlos auf den Sender warten.

Teil 1, die Einrichtung ab 040 100:

Adresse	Bytes	Befehl	Bedeutung
040 100	076 000	MVI A,000	drei Nullbytes bringen den Baustein sicher in den Grundzustand
040 102	323 373	OUT 373Q	erstes Nullbyte
040 104	323 373	OUT 373Q	zweites Nullbyte
040 106	323 373	OUT 373Q	drittes Nullbyte
040 110	076 100	MVI A,100	Befehl „interner Reset“
040 112	323 373	OUT 373Q	Reset ausführen
040 114	076 116	MVI A,116	Modus: 8 Bit, 1 Stoppbit, ohne Parität, Takt ×16
040 116	323 373	OUT 373Q	Modus setzen
040 120	076 005	MVI A,005	Kommando: Senden und Empfangen freigeben
040 122	323 373	OUT 373Q	Kommando setzen

Teil 2, die Ausgabe ab 040 124:

Adresse	Bytes	Befehl	Wirkung
040 124	041 200 040	LXI H,040 200	HL zeigt auf den Text
040 127	176	MOV A,M	nächstes Zeichen holen
040 130	267	ORA A	Zeichen null? (Textende)
040 131	312 153 040	JZ 040 153	bei null zum Ende
040 134	365	PUSH PSW	Zeichen retten
040 135	333 373	IN 373Q	Status lesen
040 137	346 001	ANI 001	Sender bereit?
040 141	312 135 040	JZ 040 135	sonst weiter warten
040 144	361	POP PSW	Zeichen zurückholen
040 145	323 372	OUT 372Q	Zeichen senden
040 147	043	INX H	nächste Textstelle
040 150	303 127 040	JMP 040 127	Schleife
040 153	166	HLT	fertig

Der Text selbst steht ab Adresse 040 200 im Speicher, in ASCII-Codes mit abschließender Null:

Adresse	200	201	202	203	204	205	206	207
Byte	110	101	114	114	117	015	012	000
Zeichen	H	A	L	L	O	CR	LF	Ende

Tippe zuerst das Programm ab 040 100 ein (44 Bytes), dann den Text ab 040 200, setze den Programmzähler auf 040 100 und drücke GO. Auf dem Terminal erscheint **HALLO** und der Cursor springt in die nächste Zeile.



Programm 5 in der Emulation: Auf dem H9 erscheint HALLO.

Programm 6: Tastatur zurückspiegeln

Das zweite Programm gibt jedes getippte Zeichen sofort wieder aus. Das ist der klassische Test für eine Terminalverbindung.

Die Einrichtung ist dieselbe wie in Programm 5 (Adressen 040 100 bis 040 123). Hast du Programm 5 schon eingetippt, bleibt sie im Speicher stehen. Ab 040 124 folgt dann die Echo-Schleife, die du über den Ausgabeteil von Programm 5 schreibst:

Adresse	Bytes	Befehl	Wirkung
040 124	333 373	IN 373Q	Status lesen
040 126	346 002	ANI 002	Zeichen empfangen?
040 130	312 124 040	JZ 040 124	nein, weiter warten
040 133	333 372	IN 372Q	Zeichen holen
040 135	365	PUSH PSW	Zeichen retten

Adresse	Bytes	Befehl	Wirkung
040 136	333 373	IN 373Q	Status lesen
040 140	346 001	ANI 001	Sender bereit?
040 142	312 136 040	JZ 040 136	nein, warten
040 145	361	POP PSW	Zeichen zurückholen
040 146	323 372	OUT 372Q	Zeichen senden
040 150	303 124 040	JMP 040 124	von vorn

Starte mit PC auf 040 100 und GO. Alles, was du am Terminal tippst, erscheint wieder auf dem Schirm. Das Programm endet nie von selbst. Beende es mit RTM (am Mac in der Emulation Umschalt + 0). Zeigt sich nichts, prüfe Baudrate, Kabel und die Einstellungen an der H8-5, bevor du am Programm zweifelst.



Programm 6 in der Emulation: Der getippte Text HEATHKIT H8 erscheint auf dem Schirm, das Programm läuft weiter.

> Hinweis: Beide Programme habe ich im Emulator „The Final H8/H9“ laufen lassen, aber nicht an echter H8-5-Hardware. Das Kommandobyte kann je nach Ausführung der Karte Anpassung brauchen. Das Handbuch zur H8-5 nennt die genauen Vorgaben.

Kapitel 6 – Kassette: Programme laden und sichern

Ohne Diskettenlaufwerk ist die Kassette der einzige Speicher des H8. Der Monitor kann sie bedienen, du brauchst dafür keinen Zusatzcode: Eine Taste lädt, eine Taste sichert.

Was du brauchst

- die Karte **H8-5** mit dem Kassetteninterface
- einen normalen Audio-Kassettenrecorder mit Eingang und Ausgang
- Kassetten mit Programmen oder leere Kassetten

Das Aufzeichnungsverfahren ist eine 1200-Baud-Variante des Kansas-City-Standards. Der Kassettenport liegt bei 370Q (Daten) und 371Q (Steuerung). Die Adresse ist im Monitor-ROM fest verdrahtet und lässt sich nicht verlegen.

Ein Programm laden

1. Spule die Kassette an den Anfang der Aufnahme und starte die Wiedergabe.
2. Wähle den Speichermodus (MEM) und drücke LOAD.
3. Der Monitor sucht den ersten Datensatz. Es dauert einen Moment, bis das Vorspannsignal der Kassette vorbei ist.
4. Danach laufen auf der Anzeige die Adresse und der gerade geladene Wert mit. Am Ende ertönt ein langer Ton, und die Endadresse bleibt stehen.
5. Der Monitor hat den Startpunkt des Programms bereits in den Programmzähler gelegt. Reagiert die Anzeige nach dem Laden nicht mehr auf Tasten, drücke einmal CANCEL. Dann zeigt REG, 6 die Startadresse, und GO startet das Programm.

Kommt statt des langen Tons ein wiederholtes Piepen, ist der Ladevorgang schiefgegangen. Mit CANCEL brichst du ab, danach lädst du neu.

Fehlernummer	Bedeutung
001	Prüfsummenfehler: Der Datensatz ist beschädigt
002	Datensatzfehler: kein Speicherabbild oder mitten im Band begonnen

Eine typische Ursache für Fehler 002: Du versuchst, einen BASIC-Programmtext oder eine Textdatei als Programm zu laden. Der Monitor lädt nur Speicherabbilder.

Ein Programm sichern

Vor dem Sichern braucht der Monitor drei Angaben:

1. den **Startpunkt**: die Adresse, bei der das Programm später anfangen soll, im Programmzähler
2. die **Anfangsadresse** der Daten: niedriges Byte an Adresse 040 000, hohes Byte an 040 001
3. die **Endadresse**: im Speichermodus angezeigt

Als Beispiel sicherst du den Zähler aus Programm 2 (Kapitel 4), der von 040 100 bis 040 110 reicht:

Schritt	Eingabe
Programmzähler	REG, **6**, ALTER, `040100`, ALTER
Anfang, niedriges Byte	MEM, `040000`, ALTER, `100`, ALTER
Anfang, hohes Byte	MEM, `040001`, ALTER, `040`, ALTER
Ende	MEM, `040110`
Aufnahme starten	Rekorder auf Aufnahme, dann DUMP

Nach der Aufnahme zeigt ein langer Ton das Ende an. Ein Kopieren von Band zu Band ist einfach: Lade das Band, lege ein leeres in den Rekorder und drücke DUMP, denn nach dem Laden stehen alle drei Angaben schon richtig im Monitor.

In der Emulation: Wähle im Kassettendeck neben dem H8 zuerst „Leeres Band“. Gib die drei Angaben aus der Tabelle ein und drücke DUMP (kurz antippen genügt, bei langem Halten schreibt der Monitor die Aufnahme zweimal). Zum Zurücklesen setzt du das Band mit „Rücklauf“ an den Anfang, drückst MEM und LOAD und danach CANCEL. Der Startpunkt kommt vom Band: REG, 6 zeigt `040 100`, auch wenn du vorher etwas anderes eingestellt hattest. Das Programm läuft wie vorher (A = `144`).

Tipps aus der Praxis

- Mach immer eine Sicherungskopie wichtiger Bänder.
- Beschrifte die Kassetten mit Programmname, Start- und Endadresse.
- Probiere die Lautstärke des Rekorders beim Laden aus, falls Prüfsummenfehler auftreten.

- Emulatoren arbeiten mit Dateien im Format H8T statt mit Kassetten. Im Browser-Emulator „The Final H8/H9“ gibt es dafür ein Kassettenlaufwerk mit Laden, Speichern und Schreibschutz.

Kapitel 7 – Benton Harbor BASIC: erste eigene Programme

Heath lieferte seinen eigenen BASIC-Dialekt mit. Er orientiert sich am Dartmouth-BASIC und unterscheidet sich deshalb von den bekannten späteren Dialekten.

Zwei Fassungen

Fassung	Eigenschaften
Benton Harbor BASIC	die einfache Fassung, ohne Textvariablen
Extended Benton Harbor BASIC	braucht mindestens 24 KByte Speicher, bietet Textvariablen, Ganzzahlen und Befehle für Disketten

Beides kam auf Kassette. Alle Beispiele in diesem Kapitel habe ich im Emulator mit der Extended-Fassung (Issue #10.02.01) laufen lassen. Die einfache Fassung meldete dort bei PRINT mit Text in Anführungszeichen den Fehler IC, deshalb sind die Beispiele für Extended geschrieben.

BASIC starten

1. Terminal anschließen und die Konsole einrichten (Kapitel 5).
2. Die BASIC-Kassette per LOAD laden (Kapitel 6).
3. GO drücken. Nach kurzer Zeit meldet sich BASIC auf dem Terminal mit einem Stern als Eingabeaufforderung.

Im Emulator „The Final H8/H9“ passiert das alles beim Laden der Seite von selbst: Der H8 lädt Extended BASIC und du siehst gleich den Stern.

Das H9 liefert nur Großbuchstaben. Schreibe deshalb alles groß.

Wichtig: BASIC ergänzt die Wörter selbst

Benton Harbor BASIC schreibt Schlüsselwörter beim Tippen aus. Du tippst die kürzeste eindeutige Abkürzung, BASIC macht das ganze Wort daraus. Schreibst du ein Wort aus, erzeugt das einen Fehler. Das ist das Originalverhalten.

Schlüsselwort	Das tippst du	Bedeutung
PRINT	`PR`	Text oder Werte ausgeben
INPUT	`INP`	Zahl von der Tastatur lesen
FOR	`FO`	Schleife beginnen
NEXT	`NE`	Schleife schließen
IF	`IF`	Bedingung
THEN	`TH`	dann springe zu Zeile
GOTO	`GOT`	Sprung
END	`EN`	Programmende
LIST	`LIS`	Programm anzeigen
RUN	`RU`	Programm starten

Die Abkürzung muss eindeutig sein: `LI` genügt für LIST nicht, erst `LIS` wird erkannt. Das Wort LET lassen wir weg. Zuweisungen schreibst du ohne Schlüsselwort (`S = 0`), denn `LE` würde zur Funktion LEN ergänzt.

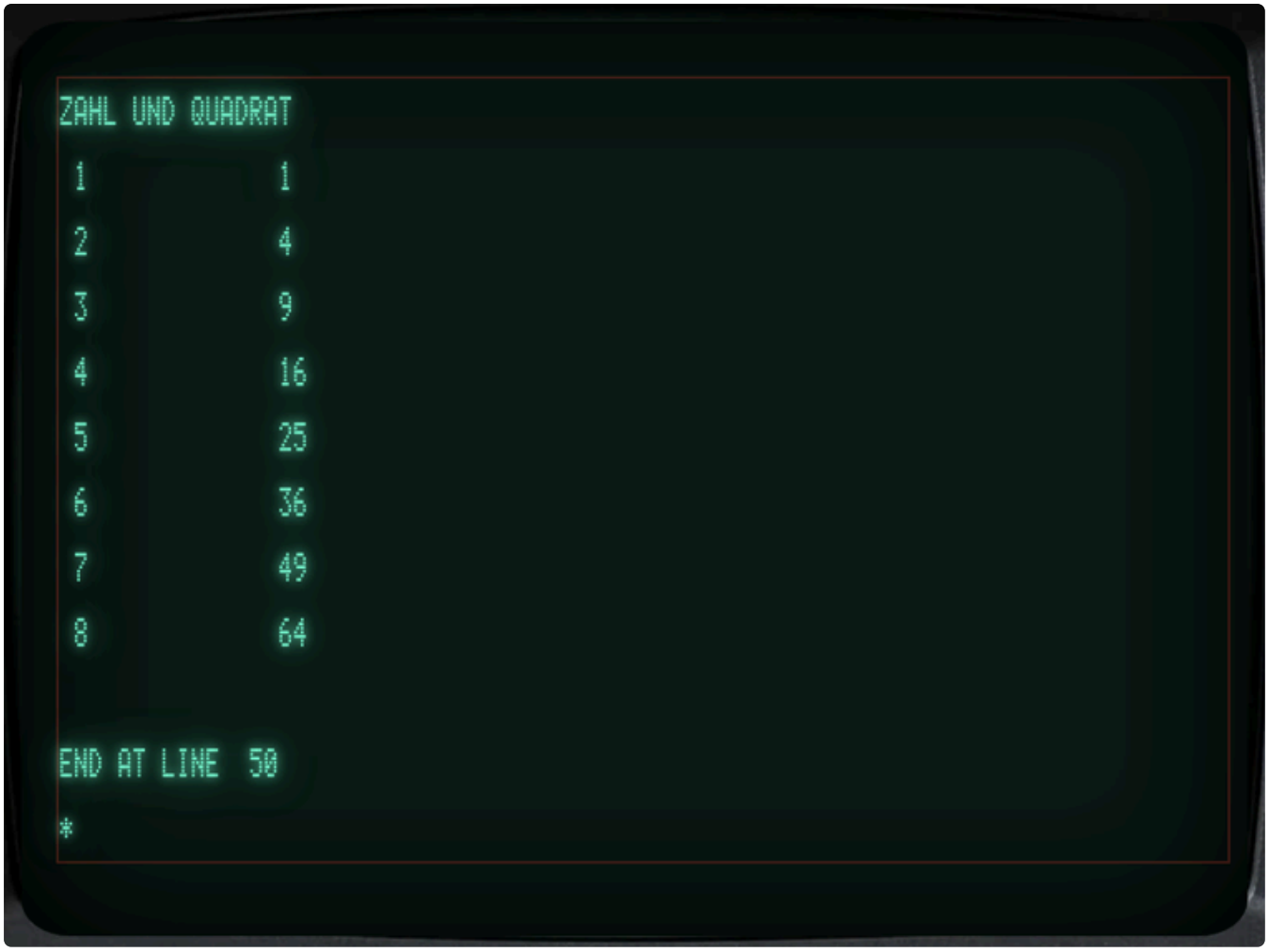
Jede Programmzeile beginnt mit einer Zeilennummer. Ohne Nummer wird die Eingabe sofort ausgeführt, mit Nummer wird sie als Programmzeile gespeichert. Nach dem Programmende meldet BASIC `END AT LINE` mit der Zeilennummer des END-Befehls.

Die Listings unten zeigen, was du tippst. Mit `LIS` siehst du das Programm danach ausgeschrieben, zum Beispiel `10 PRINT "ZAHL UND QUADRAT"`. Der H9-Schirm hat nur zwölf Zeilen und rollt weiter, wenn er voll ist. Um ein Programm zu löschen, startest du BASIC neu (im Emulator mit dem Reset-Knopf der Seite).

Programm 7: Quadratzahlen

```
10 PR "ZAHL UND QUADRAT"
20 FO I = 1 TO 8
30 PR I, I*I
40 NE I
50 EN
```

Gib `RU` ein und du bekommst eine Überschrift und acht Zeilen mit den Zahlen 1 bis 8 und ihren Quadraten.

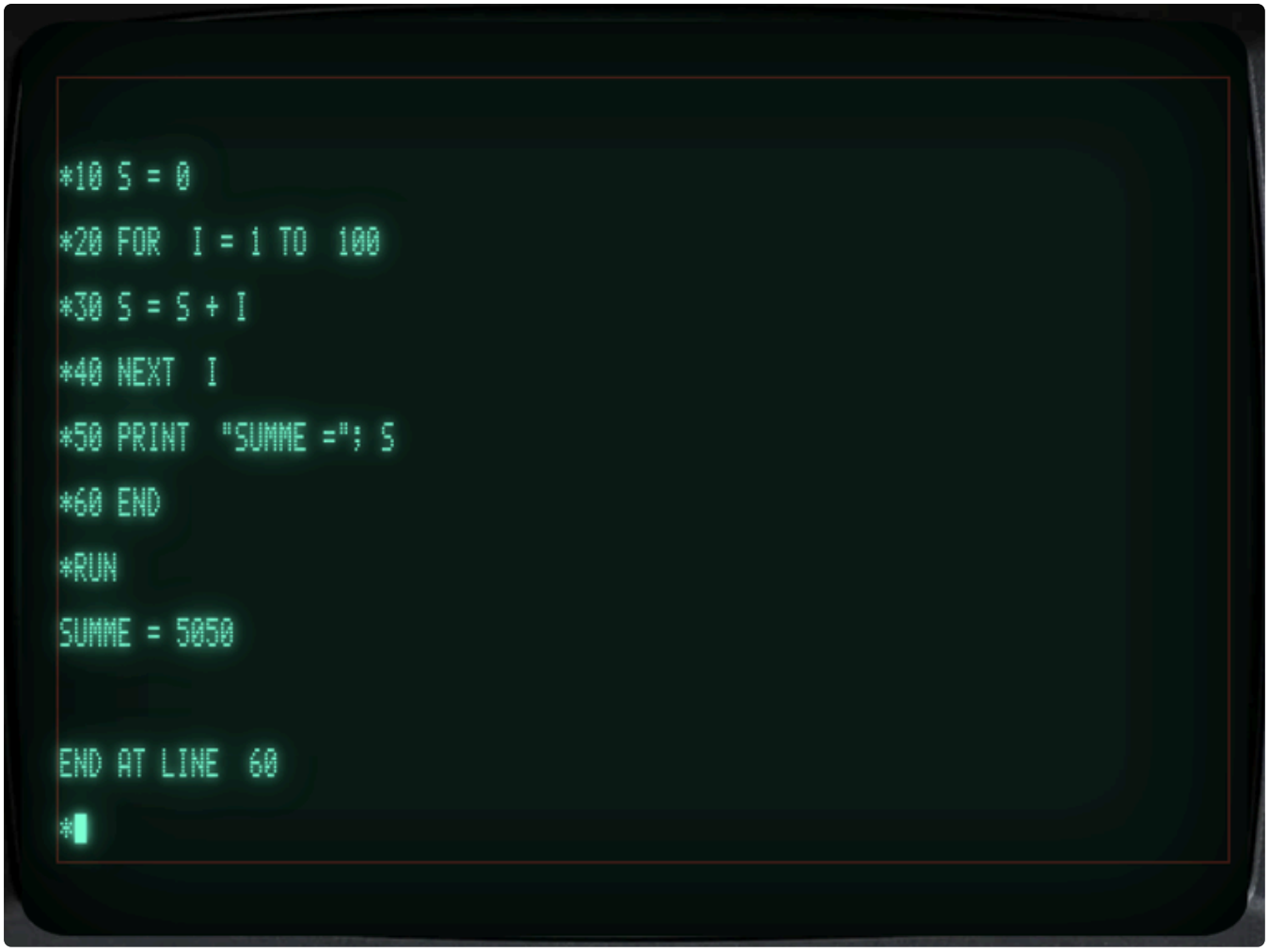


Programm 7 im Emulator: die Quadratzahlen von 1 bis 8, am Ende END AT LINE 50.

Programm 8: Summe von 1 bis 100

```
10 S = 0
20 FO I = 1 TO 100
30 S = S + I
40 NE I
50 PR "SUMME ="; S
60 EN
```

Das Ergebnis ist 5050. Vergleiche mit Programm 3 aus Kapitel 4: Dasselbe Prinzip, aber hier kümmert sich BASIC um Register und Sprünge.



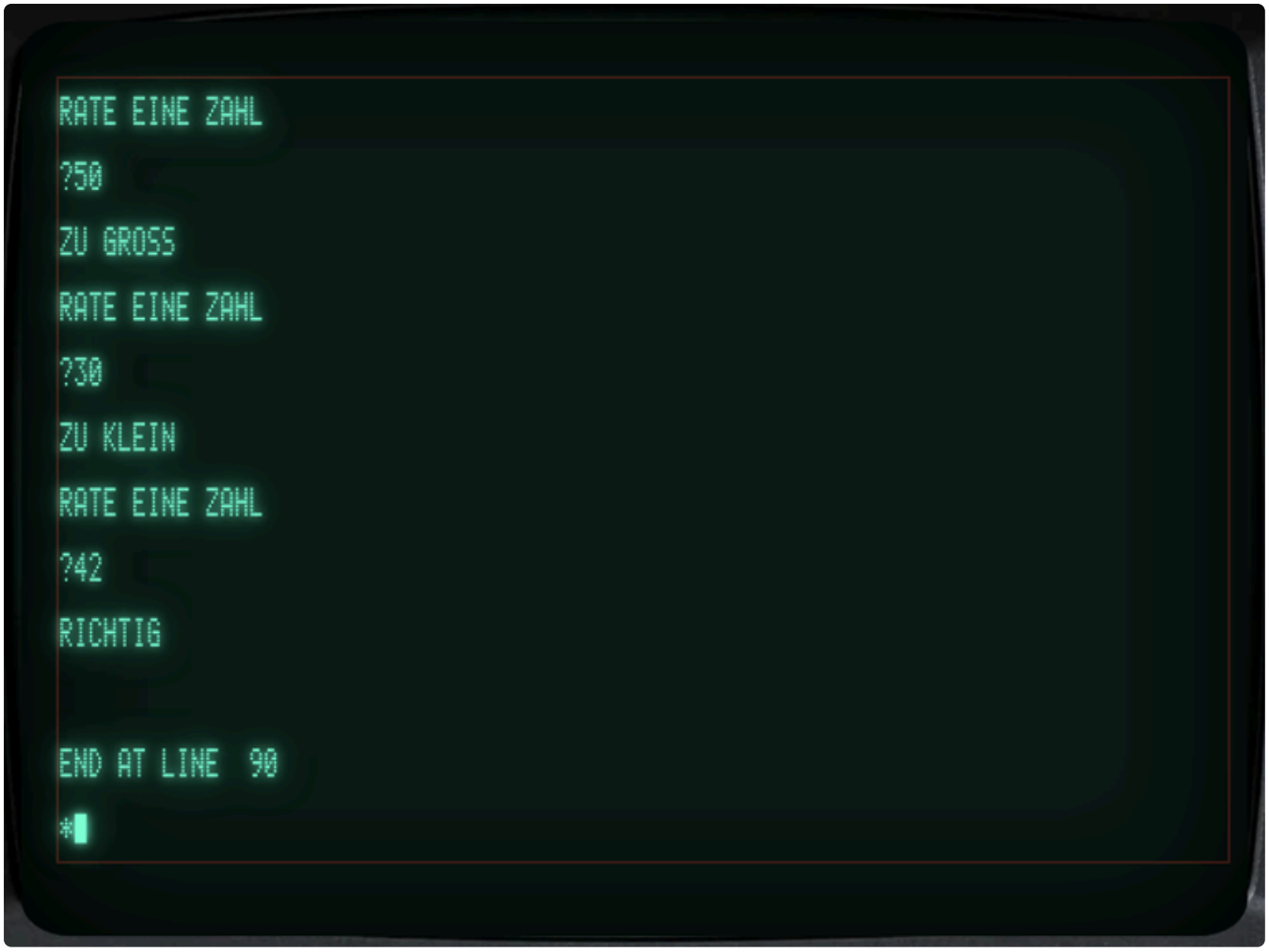
Programm 8 im Emulator: Auf dem Schirm steht das ganze Listing, darunter nach RU das Ergebnis SUMME = 5050.

Programm 9: Zahlen raten

Der Rechner denkt sich 42. Du rätst, er sagt dir, ob du zu hoch oder zu niedrig liegst.

```
10 Z = 42
20 PR "RATE EINE ZAHL"
30 INP G
40 IF G = Z TH 80
50 IF G < Z TH 70
60 PR "ZU GROSS"
65 GOT 20
70 PR "ZU KLEIN"
75 GOT 20
80 PR "RICHTIG"
90 EN
```

Beachte den Trick mit den Zeilennummern in Zehnerschritten: So bleibt Platz für spätere Einfügungen wie 65 und 75.



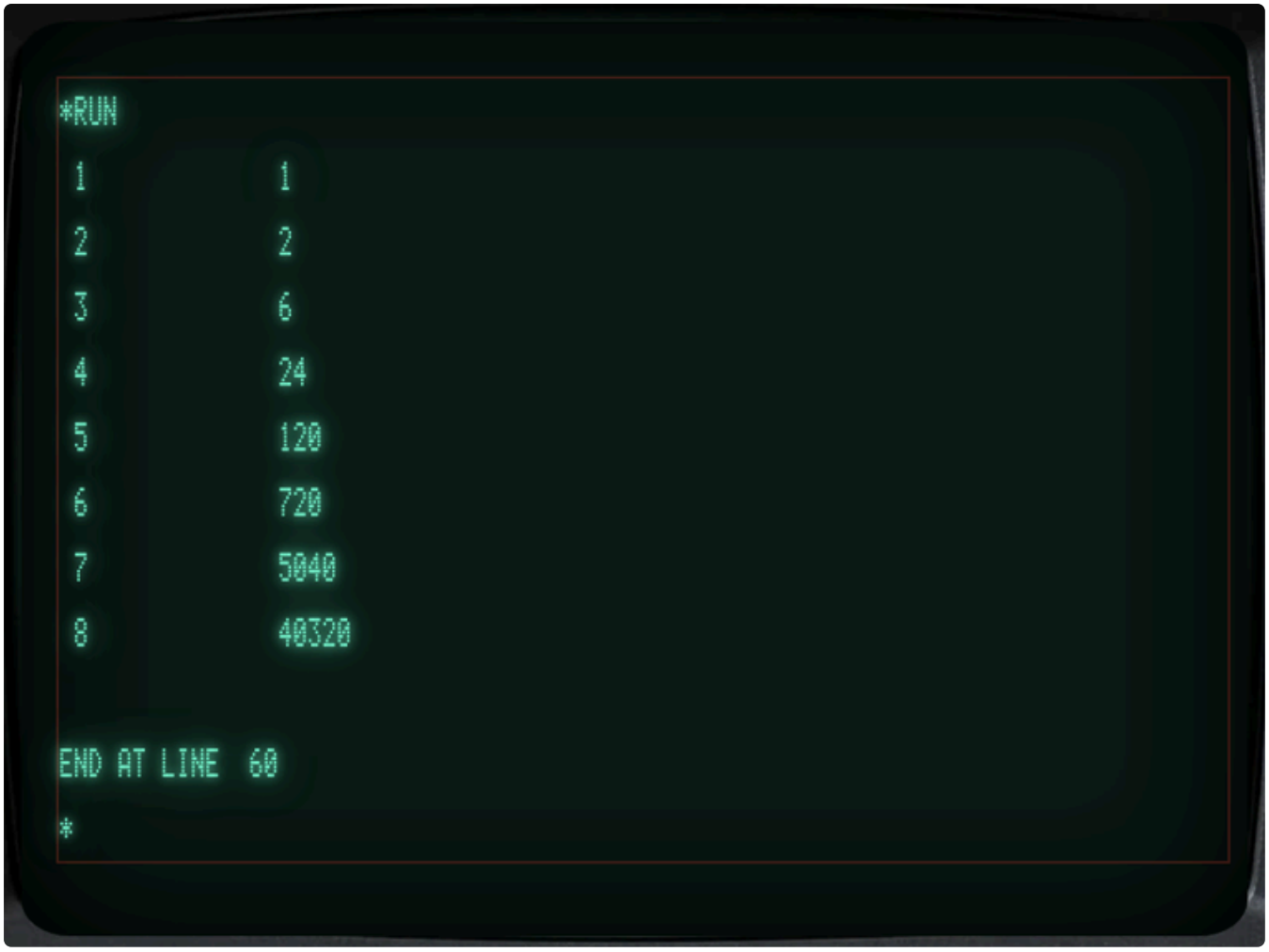
Programm 9 im Emulator: Die Eingaben 50, 30 und 42 führen zu ZU GROSS, ZU KLEIN und RICHTIG.

Programm 10: Fakultäten

Die Fakultät einer Zahl ist das Produkt aller Zahlen von 1 bis zu ihr. Das Programm gibt sie für 1 bis 8 aus:

```
10 F = 1
20 FO I = 1 TO 8
30 F = F * I
40 PR I, F
50 NE I
60 EN
```

Die Zahlen wachsen schnell: 8 Fakultät ist schon 40320.



Programm 10 im Emulator: Fakultäten von 1 bis 8.

Übungen

1. Ändere Programm 7 so, dass es die Kubikzahlen ausgibt.
2. Erweitere Programm 9 um einen Zähler, der die Versuche mitzählt.
3. Ändere in Programm 10 die Obergrenze auf 10 und schau dir an, wie BASIC die Zahl 3628800 darstellt (in meinem Test als 3.62880E+06).

Kapitel 8 – 8080-Assembler für Einsteiger

In Kapitel 4 hast du Programme als Oktalzahlen eingetippt. Das ist Maschinensprache. Assembler ist dieselbe Sprache mit lesbaren Namen: statt `200` schreibst du `ADD B`. Ein Übersetzungsprogramm, der Assembler, macht daraus wieder die Zahlen.

Die Register des 8080

Register	Aufgabe
A	Akkumulator, Ziel fast aller Rechenbefehle
B, C, D, E, H, L	allgemeine 8-Bit-Register
BC, DE, HL	Registerpaare für 16-Bit-Werte und Adressen
M	kein Register: der Speicher, auf den HL zeigt
SP	Stapelzeiger
PC	Programmzähler

Dazu kommen fünf Flags, die bei Rechenbefehlen gesetzt werden: Zero (Ergebnis null), Sign (Ergebnis negativ), Parity, Carry (Übertrag) und Auxiliary Carry.

Die Befehlsgruppen

- **Daten verschieben:** MOV, MVI, LXI, LDA, STA, LDAX, STAX, XCHG
- **Rechnen:** ADD, ADI, ADC, SUB, SUI, SBB, INR, DCR, INX, DCX, DAD
- **Logik und Vergleich:** ANA, ORA, XRA, CMP, CPI, Rotationen
- **Sprünge und Unterprogramme:** JMP, bedingte Sprünge (JZ, JNZ, JC, JNC …), CALL, RET, RST
- **Stapel, Ein- und Ausgabe, Steuerung:** PUSH, POP, IN, OUT, EI, DI, HLT, NOP

Die Oktalcodes der Befehle aus diesem Buch stehen im Anhang.

Wie ein Befehl aufgebaut ist

Ein 8080-Befehl hat ein, zwei oder drei Bytes. Das erste Byte nennt die Operation. Ein Zusatzbyte trägt einen Wert, zwei Zusatzbytes tragen eine Adresse mit dem niedrigen Byte zuerst.

Das Schöne an Oktal: Der Opcode lässt sich ablesen. Beim Befehl `MOV Ziel,Quelle` besteht der Code aus einer 1 und zwei Registernummern:

Register	B	C	D	E	H	L	M	A
Nummer	0	1	2	3	4	5	6	7

`MOV A,B` ist darum `1`, dann Ziel A (7), dann Quelle B (0): **170**. Und `MVI B,n` ist `0`, Ziel B (0), `6`: **006**.

Dieselbe Schleife in Assembler

Programm 3 aus Kapitel 4 sieht als Quelltext so aus:

```
        ORG   40100A
START   MVI   B,10
        MVI   A,0
LOOP    ADD   B
        DCR   B
        JNZ   LOOP
        HLT
```

Im Heath-Assembler steht ein nachgestelltes **A** für eine Oktalzahl, ein **B** für eine Binärzahl. Zahlen ohne Zusatz sind dezimal. `LOOP` ist eine **Marke**: Der Assembler rechnet selbst aus, an welcher Adresse sie steht, du musst das nicht mehr von Hand tun.

Programm 4 (Speicher füllen):

```
        ORG   40100A
START   LXI   H,40200A
        MVI   B,16
        MVI   A,377A
LOOP    MOV   M,A
        INX   H
        DCR   B
        JNZ   LOOP
        HLT
```

Von Hand assemblieren

Auch ohne Assembler kommst du zum Ziel. So gehst du vor:

1. Schreibe das Programm als Befehlsliste auf Papier.
2. Schlage zu jedem Befehl den Oktalcode nach.
3. Vergib Adressen. Zähle dabei die Bytes jedes Befehls (1, 2 oder 3).
4. Setze bei Sprüngen die Zieladresse ein, niedriges Byte zuerst.
5. Tippe alles über die Frontplatte ein.

Aufgabe: Speicherbereich kopieren

Kopiere 16 Bytes von 040 200 nach 040 220. Du brauchst zwei Zeiger: HL für die Quelle und DE für das Ziel. Die Befehle `MOV A,M`, `STAX D`, `INX H`, `INX D` und eine Schleife mit `DCR B` und `JNZ` genügen. Schreibe das Programm zuerst als Quelltext, dann als Oktalcodes, und probiere es aus.

Eine Lösung (im Emulator getestet; die Bytes stehen ab 040 100):

```
        ORG   40100A
START   LXI   H,40200A     041 200 040
        LXI   D,40220A     021 220 040
        MVI   B,16         006 020
LOOP    MOV   A,M          176
        STAX  D            022
        INX   H            043
        INX   D            023
        DCR   B            005
        JNZ   LOOP         302 110 040
        HLT                166
```

Schreibe vor dem Start 16 beliebige Bytes ab 040 200. Nach GO und dem Halt stehen dieselben Bytes ab 040 220.

Unterprogramme und der Stapel

Mit `CALL` springst du in ein Unterprogramm, mit `RET` kehrst du zurück. Der Rechner merkt sich die Rückkehradresse auf dem **Stapel**. Der Stapelzeiger SP steht beim Einschalten am oberen Ende des Speichers. Beachte, dass der Monitor selbst auch den Stapel nutzt (Kapitel 3): Lass immer genug Platz.

Kapitel 9 – Disketten und HDOS

Kassetten sind billig, aber langsam und launisch. Wer den H8 ernsthaft nutzen will, rüstet früher oder später ein Diskettenlaufwerk nach.

Das Laufwerk H17

Das Laufwerk H17 ist ein 5¼-Zoll-Laufwerk mit hartsektorierten Disketten. Es fasst rund 100 KByte pro Diskette. Der Controller sitzt auf einer Steckkarte im H8 und belegt die Ports 174Q bis 177Q (Kapitel 3).

HDOS

Das **Heath Disk Operating System** (HDOS) stammt von Gordon Letwin. Es kam mit dem H17 und bietet:

- Dateien mit Namen statt Kassetten mit Beschriftung
- einen Befehlsinterpreter, in den du Befehle wie `DIR` tippst, um das Inhaltsverzeichnis zu sehen

- Gerätetreiber, sodass Programme auf Konsole, Drucker oder Laufwerk zugreifen, ohne die Ports zu kennen
- einen Editor und einen Assembler, mit denen du Quelltexte wie in Kapitel 8 am Rechner schreibst und übersetzt

Das Betriebssystem braucht mehr Speicher als der Grundausbau. Plane mindestens 16 KByte RAM ein, komfortabler sind 32 KByte.

Starten von Diskette

Das Starten (Booten) erfolgt über den Monitor. Je nach Controller-Karte und ROM-Version unterscheidet sich der Vorgang. Schlage die genaue Tastenfolge im Handbuch deiner Karte nach.

CP/M als Alternative

Wer lieber das weit verbreitete CP/M nutzen möchte, braucht dafür eine passende Anpassung an die H8-Hardware und ein dafür geeignetes Laufwerk. Der Vorteil: Eine riesige Menge an Software läuft darauf. Der Nachteil: Sie ist auf H8-Hardware oft schwerer zu beschaffen als HDOS.

Was bleibt gleich

Auch mit HDOS gilt alles aus den vorigen Kapiteln: Die Frontplatte bleibt als Notausgang, der Monitor sitzt weiter im ROM, und ein Absturz lässt sich mit ø+RST beenden.

Kapitel 10 – Fehlersuche

Der H8 ist ein selbst gebauter Rechner mit Steckkarten, Kontakten und Netzteil. Wenn etwas nicht geht, ist die Ursache meist banal. Gehe der Reihe nach vor.

Nichts passiert beim Einschalten

1. Netzspannung und Sicherung prüfen.
2. Alle Karten sitzen richtig in ihren Steckplätzen? Kontakte vorsichtig reinigen.
3. Steckt die CPU-Karte in P2 und die Frontplatte in P1?
4. Netzteilspannungen messen, falls Messgerät vorhanden.

Die Anzeige zeigt Unsinn

Zieh den Rechner mit ø+RST zurück in einen definierten Zustand. Zeigt die Frontplatte danach weiter Unsinn, liegt der Fehler meist im Monitor-ROM oder an der CPU-Karte.

Programm läuft nicht wie erwartet

Symptom	Mögliche Ursache
Programm tut gar nichts	Falsche Startadresse, PC nicht gesetzt
Rechner scheint zu hängen	Endlosschleife, Sprungziel um ein Byte falsch
Anzeige springt in den Monitor zurück	HLT erreicht oder Befehl aus Versehen als Daten gelesen
Ergebnis falsch	Oktalziffer vertippt, niedriges und hohes Byte vertauscht
Absturz nach CALL	Stapel zeigt in nicht vorhandenen Speicher

Unbedingt prüfen: Stehen die Bytes wirklich dort, wo du sie vermutest? Gehe mit MEM und +/− durch das Programm und vergleiche mit deinem Zettel.

Hilfsmittel: HLT als Haltepunkt

Setze ein HLT (166) an die Stelle, an der du den Rechnerzustand sehen willst. Nach dem Halt kannst du mit REG die Register ablesen. Danach ersetzt du das HLT wieder durch den ursprünglichen Befehl.

Die Konsole schweigt

- Baudrate und Format am Terminal stimmen mit der H8-5-Einstellung überein?
- Kabel richtig herum und fest gesteckt?
- Wurde der 8251 initialisiert (Kapitel 5)? Beachte die Hinweise zum Kommandobyte.

Kassettenfehler

Vergleiche die Tabelle in Kapitel 6. Prüfsummenfehler lassen sich oft durch Neueinstellen der Lautstärke oder einen sauberen Tonkopf beheben.

Speicher prüfen

Bei unerklärlichen Fehlern liegt der Verdacht beim RAM. Schreibe ein Testmuster mit einem kleinen Programm (ähnlich Programm 4 in Kapitel 4) in einen Bereich und lies es zurück. Wechselnde Muster wie 252Q und 125Q entlarven defekte Bits besonders gut.

Kapitel 11 – Der H8 heute

Der H8 ist seit Jahrzehnten nicht mehr im Handel, aber er lebt weiter: bei Sammlern, in Foren und in Emulatoren.

Original oder Emulator?

Ein echter H8 hat Vorteile, die kein Emulator bietet: das Klicken der Tasten, das Leuchten der Segmentanzeigen und das Gefühl, selbst Hand an die Hardware zu legen. Dafür sind Teile selten, die Netzteile alt und die Platzansprüche beträchtlich.

Ein Emulator läuft dagegen auf jedem Rechner, ist sofort startklar und verzeiht Fehler.

Mein Projekt: The Final H8/H9

Wenn du die Beispiele aus diesem Buch ausprobieren möchtest, ohne einen H8 zu besitzen, hilft dir mein Projekt **The Final H8/H9**. Es ist ein Emulator, der direkt im Browser läuft, du brauchst also nichts zu installieren.

So nutzt du ihn zu diesem Buch:

1. Öffne die Seite im Browser. Mit dem Knopf „Seite 1 · H8“ siehst du den H8, mit „Seite 2 · H9“ das Terminal.
2. Bediene die Frontplatte wie in Kapitel 3 beschrieben.
3. Tippe Programm 1 aus Kapitel 4 ein und starte es mit GO.
4. Probiere Kassettenbetrieb (Kapitel 6) und Konsole (Kapitel 5) aus.

Die Maschinenprogramme 1 bis 6 (Kapitel 4 und 5), die BASIC-Programme 7 bis 10 (Kapitel 7), das Sichern und Laden auf ein leeres Band (Kapitel 6) und das Kopierprogramm aus Kapitel 8 habe ich im Emulator laufen lassen. Die Bilder dazu stammen von dort. Nicht geprüft habe ich den Heath-Assembler selbst: Die Quelltexte in Kapitel 8 sind von Hand in Oktalcodes übersetzt, und die Codes stimmen mit den getesteten Programmen überein. Auch das Laden eines BASIC-Bands von Hand habe ich nicht durchgespielt. Auf der Seite findest du, was der Emulator im Einzelnen kann.

Wichtiger Hinweis zur H9-Seite: Wenn du auf der Seite des H9-Terminals bist, kommst du mit der Taste **LINE** auf der Tastatur des Terminals zurück zum H8.

Weiterführende Quellen

- Die originalen Heath-Handbücher (Operation Manual, PAM-8 Front Panel Monitor) liegen als Scans in Archiven.
- Ein guter Einstieg in die Geschichte sind die Wikipedia-Artikel zu H8 und HDOS.
- Viele Hobbyisten tauschen sich in Foren zum Thema Heathkit aus.

Schlusswort

Der H8 zwingt dich, den Rechner zu verstehen: jedes Byte, jede Adresse, jedes Signal. Genau das macht ihn zu einem so guten Lehrer. Viel Spaß beim Ausprobieren!

Anhang – Befehlstabelle und Quellen

Wichtige 8080-Befehle in Oktal

Befehl	Oktalcode	Bytes
NOP	000	1
HLT	166	1
MOV Ziel,Quelle	1ZQ (Registernummern B0 C1 D2 E3 H4 L5 M6 A7)	1
MVI r,n	0r6 n	2
LXI B / D / H / SP	001 / 021 / 041 / 061, dann niedriges und hohes Byte	3
LDA / STA adr	072 / 062	3
LDAX B / D	012 / 032	1
STAX B / D	002 / 022	1
XCHG	353	1
ADD / ADC / SUB / SBB r	20r / 21r / 22r / 23r	1
ANA / XRA / ORA / CMP r	24r / 25r / 26r / 27r	1
ADI / SUI / CPI n	306 / 326 / 376	2
INR r / DCR r	0r4 / 0r5	1
INX B / D / H / SP	003 / 023 / 043 / 063	1
DCX B / D / H / SP	013 / 033 / 053 / 073	1
DAD B / D / H / SP	011 / 031 / 051 / 071	1
JMP adr	303	3
JNZ / JZ adr	302 / 312	3
JNC / JC adr	322 / 332	3
CALL adr / RET	315 / 311	3 / 1

Befehl	Oktalcode	Bytes
PUSH B / D / H / PSW	305 / 325 / 345 / 365	1
POP B / D / H / PSW	301 / 321 / 341 / 361	1
IN / OUT port	333 / 323	2
EI / DI	373 / 363	1
RST n	3n7	1

Ports im Überblick

Port	Bedeutung
360Q / 361Q	Frontplatte
370Q / 371Q	Kassette (Laden und Sichern)
372Q / 373Q	Konsole (8251: Daten / Status und Steuerung)
350Q–357Q	H8-4
174Q–177Q	H17 (Diskette)

Quellen

- Heathkit H8 – Wikipedia
- HDOS – Wikipedia
- H8 Operation Manual (595-2014-02), Scan bei deramp.com
- H8 Front Panel Monitor PAM-8 (595-2348), Scan bei Bitsavers
- The Final H8/H9 – Browser-Emulator